RAPPORT DE TP : mi-guidé POO python

RÉALISÉ PAR :

- ABDENBI MEHDI

ENCADRÉ PAR :

M . SARA AHSAIN

ANNÉE UNIVERSITAIRE :

2025 – 2026

1. Description du problème traité:

L'objectif de ce TP est de concevoir et développer un système de gestion de commandes pour un café en utilisant la Programmation Orientée Objet (POO) en Python. Le système doit permettre de modéliser des boissons génériques, de leur ajouter dynamiquement des ingrédients supplémentaires (lait, sucre, caramel), et de pouvoir combiner plusieurs boissons ensemble. De plus, le programme gère des clients et différents types de commandes (sur place, à emporter) tout en intégrant un système de fidélité récompensant les achats. L'enjeu principal est de créer un code flexible, modulaire et facilement évolutif.

2. Explication de l'architecture choisie:

L'architecture du projet repose fortement sur les concepts avancés de la POO :

- **Classes abstraites :** La classe de base Boisson est abstraite (hérite de ABC). Elle force toutes les boissons concrètes à implémenter les méthodes cout() et description().
- **Design Pattern "Décorateur" :** Pour éviter une explosion combinatoire des classes (ex: CafeLaitSucre), nous utilisons une classe DecorateurBoisson. Les classes Lait, Sucre et Caramel héritent de ce décorateur pour "envelopper" une boisson et modifier dynamiquement son prix et sa description.
- **Méthodes magiques (Surcharge d'opérateur) :** La redéfinition de la méthode __add__ dans la classe Boisson permet de combiner deux boissons de manière intuitive avec l'opérateur +.
- **Dataclasses :** La classe Client utilise le décorateur @dataclass pour simplifier la création d'objets destinés à stocker des données (nom, numéro, points de fidélité).
- **Héritage et Polymorphisme :** La classe Commande sert de classe parente. Les classes CommandeSurPlace et CommandeEmporter en héritent et redéfinissent (polymorphisme) la méthode d'affichage pour inclure, par exemple, des frais d'emballage.
- **Héritage multiple :** La classe CommandeFidele hérite à la fois de Commande (pour la gestion du panier) et de Fidelite (pour l'ajout de points), démontrant la capacité de Python à combiner plusieurs comportements

3. Description des principales fonctionnalités implémentées:

- **Création de boissons de base :** Instanciation de cafés et de thés avec leurs prix par défaut.
- **Personnalisation des boissons :** Ajout d'ingrédients à la volée qui mettent à jour la description et le prix total de la boisson.
- **Combinaison de boissons :** Fusion de deux objets Boisson en un seul (ex: un mix Café + Thé).
- **Gestion du panier client** : Ajout itératif de boissons dans une commande rattachée à un client spécifique.
- **Calculs et affichage différenciés** : Calcul du montant total avec prise en compte des spécificités de la commande (ex: frais fixes pour la vente à emporter).
- **Validation et fidélisation :** Lors de la validation de la commande, le montant payé est converti en points qui s'ajoutent automatiquement au solde de fidélité du client.

4. Exemples d'exécution:

```
=== TICKET DE CAISSE : mehdi abdenbi ===
 > Café noir sucré nappage caramel : 22.0 DH
 > Thé vert au lait : 14.0 DH
 > Café noir mixé avec Thé vert : 30.0 DH
-----------------------------------
MONTANT TOTAL : 66.0 DH

Validation en cours...
*** 66 points de fidélité crédités. Nouveau solde : 266 pts ***

-> Statut : Commande validée avec succès.
```

5. Lien vers le dépôt GitHub:

https://github.com/mehdizono/m361-python-/tree/main/TP2